By storing the same key in two or more different Bloom filters, possibly with different buffer size, but most importantly each one with a different set of hash functions, we can arbitrarily lower the probability of false positives.

Of course, this is not for free, so the space needed grows proportionally, and the time needed to store or check for a key in theory would double as well.

At least for the running time, though, there is a silver lining: each of the component filters can actually be queried in parallel on a multicore hardware! So, besides maintaining the `O(k)` constant time bound, even actual implementations could possibly be as fast as regular Bloom filters (in other words, the constant factor remains approximately the same).

The way this structures work is the following: for `insert`, each of the components stores the key independently. For calls to `contains`, the answer is the combination of all answers: `true` is returned only if all the components return `true`.

What's the accuracy of such an array of filters? It can be proven that the precision of a single Bloom filter using `m` bits is the same as the precision of `j` Bloom filters using `m/j` bits each.

However, by using a parallel version of the ensemble algorithm the running time is just a fraction of the original one, exactly `1/j`.

4.11.3 Layered Bloom Filter